System-One RSI: Learning External Muscle Memory for Language-Model Agents through OODA Interaction

Qingxu Fu[1]
qingxu.fu@outlook.com

Xuanfa Jin[1,2]
xuanfa.jin@alibaba-inc.com

Zhaoyang Liu[1]
liuzhaoyang@alibaba-inc.com

Bolin Ding[1]
dingbolin@alibaba-inc.com

[1]Alibaba Token Foundry - Infra & Model RSI - EconML

[2]Institute of Automation, Chinese Academy of Sciences

November 2026

Abstract

Task knowledge alone does not establish reliable control in dynamic environments. System-One Recursive Self-Improvement (System-One RSI) converts task knowledge and interaction experience into reusable visual judgments. A reasoning model supplies control rules and supervision through informal language, guiding targeted practice in the environment. JEV stores learned distinctions in its parameters and returns structured judgments without generating a reasoning trace at every step. An Observe, Orient, Decide, Act (OODA) controller uses these judgments to select actions. Recorded experience supports subsequent model training and controller revision without requiring expert operational demonstrations. Deployments in *Zenless Zone Zero* and *Black Myth: Wukong* examine prediction accuracy and online behaviour. The studies include visual-model adaptation and controller revision across opponents. The framework targets environments where repeated interaction is costly and parallel simulation is impractical. Its application depends on task knowledge, the coverage of collected experience, and the resources available for inference.

1 Introduction

Reasoning about a task does not by itself establish the ability to perform it reliably in a changing environment. Interaction requires accurate state assessment and action selection within the available response interval. Invoking extended reasoning at each step can exceed this response budget, motivating architectures that separate rapid decisions from deliberate planning [1, 2]. A central learning problem is therefore to consolidate task knowledge and interaction experience into reusable judgments that can be invoked without repeated deliberation.

JEV provides a structured, non-generative decision interface [3], while Jeva develops decision-native representations for variable candidate sets [4]. Its non-generative outputs support rapid inference, while adaptation to unfamiliar environments requires task-specific supervision and experience. Knowledge and policy distillation provide mechanisms for transferring reference-model behaviour into learned parameters [5, 6], and rationale-based supervision extends this idea to reasoning models [7]. Our focus is on connecting such knowledge transfer to the states visited by a live agent and to the controller that uses its judgments.

We view the resulting JEV parameters as the agent's external *muscle memory*. Knowing the principles of cycling does not establish the ability to ride. Research on deliberate practice distinguishes acquiring expert performance from merely accumulating knowledge [8]. Our analogy emphasizes targeted practice under a teacher's informal verbal guidance to connect knowledge to sensorimotor responses. In System-One RSI, a reasoning model provides this guidance. Prior knowledge can guide what to practice, so capability formation need not rely exclusively on undirected trial and error. The analogy emphasizes rapid recognition and response without repeated deliberation, consistent with the fast/slow distinction [9] and the separation of semantic knowledge from executable skills [10].

Figure 1 relates guided human practice to the proposed learning process. The upper row uses swimming to illustrate how verbal guidance and practice contribute to a motor skill. The lower row maps prior knowledge and interaction-based training to a JEV model used for real-time judgments.

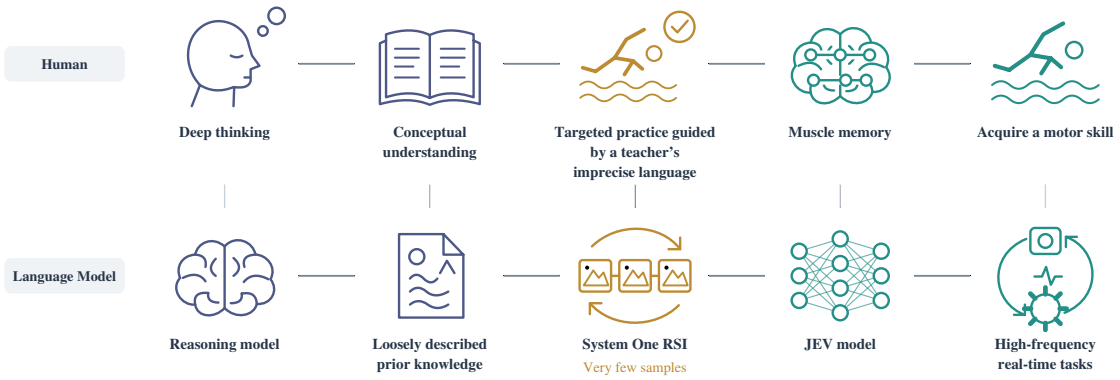


Figure 1: Guided practice and external muscle memory.

This analogy motivates the separation of knowledge, practice, and online response. The following alternatives differ in how they acquire and retain the judgments required for control.

Online reinforcement learning (RL) acquires policies through repeated interaction [11], but online accounts, authentication, persistent state, and external service dependencies can make a live environment difficult to replicate, reset, or execute in parallel. These deployment constraints are instances of the gap between standard RL assumptions and real-world systems [12]. They

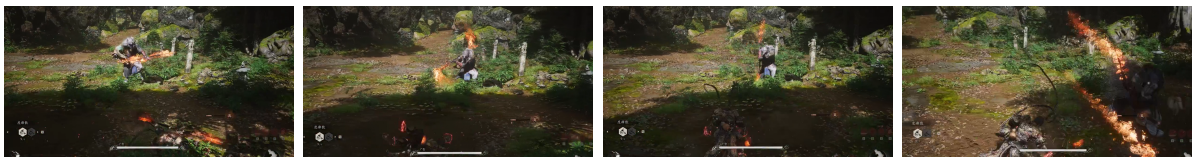
limit the interaction throughput available for training and make prior knowledge particularly valuable when only a small number of serial trials is feasible. Offline reinforcement learning instead learns from previously collected trajectories, with performance constrained by data coverage and distribution shift [13]. Obtaining sufficiently informative trajectories can itself be difficult in nonstandard environments. Learning from task supervision and retained experience offers a way to adapt without first constructing a parallel rollout sandbox.

Table 1 connects representative approaches to the obstacles considered here. Its limitations column applies these mechanisms to the live-environment setting of this paper. A reasoning model can generate programs combining computer vision (CV) with OODA control [14, 15], but enumerating visual conditions and exceptional states creates a substantial testing and maintenance burden. Prompt design changes how a pretrained model is queried without necessarily updating its parameters [16]. Compact prompts can omit relevant distinctions, while detailed descriptions and reference images increase the input that must be processed. Computer-use evaluations also identify visual grounding and operational knowledge as obstacles to successful execution [17]. The temporal example below illustrates a related difficulty in specifying action-relevant distinctions through language.

Table 1: Approaches to acquiring control capabilities in dynamic environments.

Approach	Advantages	Limitations
Online RL [11, 12]	Learns policies from interaction feedback	Accounts, authentication, and external services obstruct replication, reset, and parallel rollouts.
Offline RL [13]	Reuses previously collected trajectories	Limited coverage and distribution shift constrain what can be learned from the available data.
Reasoning-generated CV/OODA [14, 15]	Converts prior knowledge into fast executable rules	Visual edge cases require extensive testing and maintenance; new experience does not automatically improve perception.
Reasoning-based computer use [1, 17]	Uses semantic knowledge and flexible task decomposition	Visual grounding and operational knowledge remain obstacles; repeated reasoning adds to the response path.
Unadapted JEV [2, 3]	Produces compact structured judgments	A rapid decision interface does not establish competence in unfamiliar task states.
JEV prompt optimization [3, 16]	Revises task instructions without parameter updates	Fine visual distinctions can resist verbal specification; longer inputs increase online processing cost.

Figure 2 shows why temporal judgment requires more than a coarse description of posture. The second and third observations both show a raised weapon, but occur at different stages before the attack.



(a) (b) (c) (d)

Figure 2: Successive observations of an attack in Black Myth: Wukong.

Distinguishing panels (b) and (c) requires attention to weapon angle and the recent change in posture. A short description can merge these states, while a longer description must still identify

the visual features that determine action timing. Learning from observation sequences provides a mechanism for encoding such distinctions in parameters rather than repeatedly describing them in the prompt.

Self-improvement methods differ in the object they update. Reflexion retains linguistic feedback in episodic memory [18]; SICA and the Darwin Gödel Machine modify agent code [19, 20]; self-referential weight matrices learn runtime weight-update mechanisms [21]. We use a reasoning model to supervise JEV adaptation and revise its executable controller. The training procedure remains fixed. This formulation connects parameter-based skill acquisition to feedback-driven agent improvement.

System-One RSI connects task supervision to repeated interaction. The online OODA loop uses JEV judgments and executable rules to control the environment. The outer loop uses recorded experience and a multimodal reasoning model to define supervision, adapt the JEV model, and revise the controller before returning the system to interaction. Supervision can take the form of annotations from a reasoning model or a label-generating protocol specified by the reasoning model. The deployed agent influences the states collected for later learning, connecting this loop to supervision on learner-visited states [22]. This dependence of successive training data on the deployed agent defines the recursive improvement process.

The framework supports improving JEV alone, the OODA controller alone, or both. This separation allows the reasoning model to revise explicit rules or consolidate difficult perceptual distinctions into parameters, according to the capability that requires improvement. The deployments examine visual judgment and online control in Zenless Zone Zero and Black Myth: Wukong.

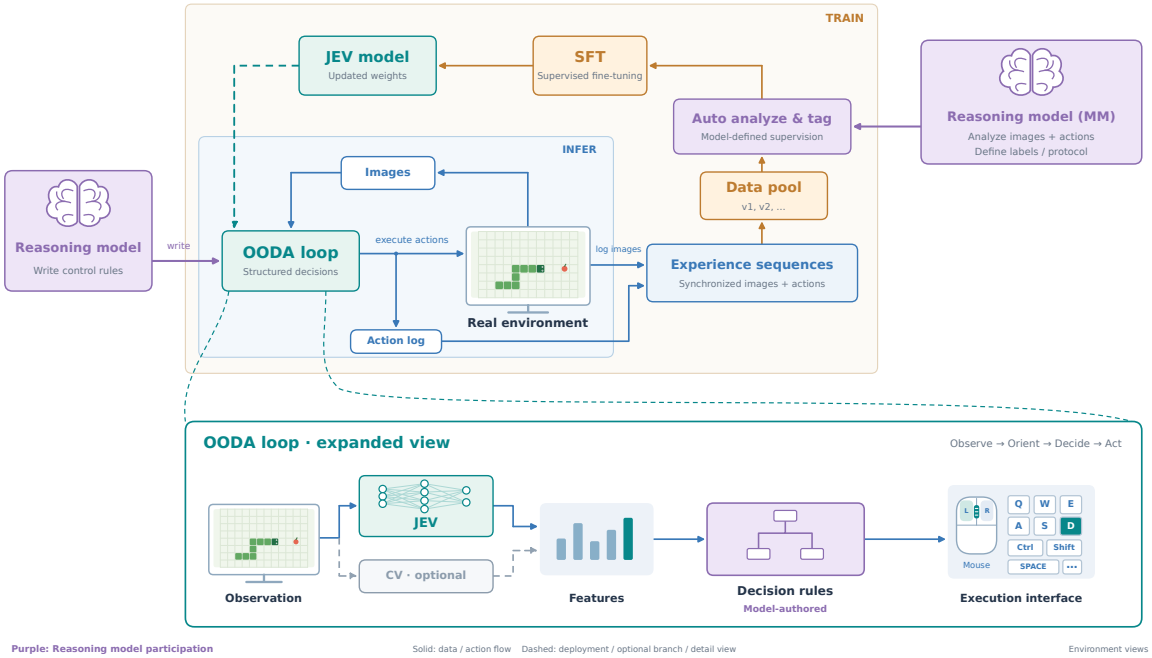


Figure 3: Interaction and adaptation in System-One RSI.

Figure 3 traces observations and actions from online interaction into the experience pool. The reasoning model uses these records to define supervision and revise control rules. The expanded panel shows how the resulting judgments enter the online action path.

2 The System-One RSI Pipeline

System-One RSI comprises an online control loop and an offline adaptation loop. During interaction, JEV supplies structured judgments to an OODA controller; between interaction rounds, a reasoning model uses recorded experience to define supervision and revise the model, the controller, or both. The deployed system generates experience, and the adapted system returns to the environment. The implementation combines a pretrained visual backbone, a structured decision interface, low-rank adaptation (LoRA) [23], and an executable controller. It uses existing hardware and model architectures, with achievable response time determined by the serving implementation and deployment conditions.

2.1 Coordinating the adaptation workflow

The implementation can distribute controller development, interaction capture, supervision, and model training among specialized agents (Figure 4). A coordinator maintains the task specification and connects these activities through shared interaction records and deployment artifacts. This organization supports the two update paths while preserving an explicit interface between judgments and actions. The coordinator connects rule development to data collection and training, so each update returns to the same execution interface.

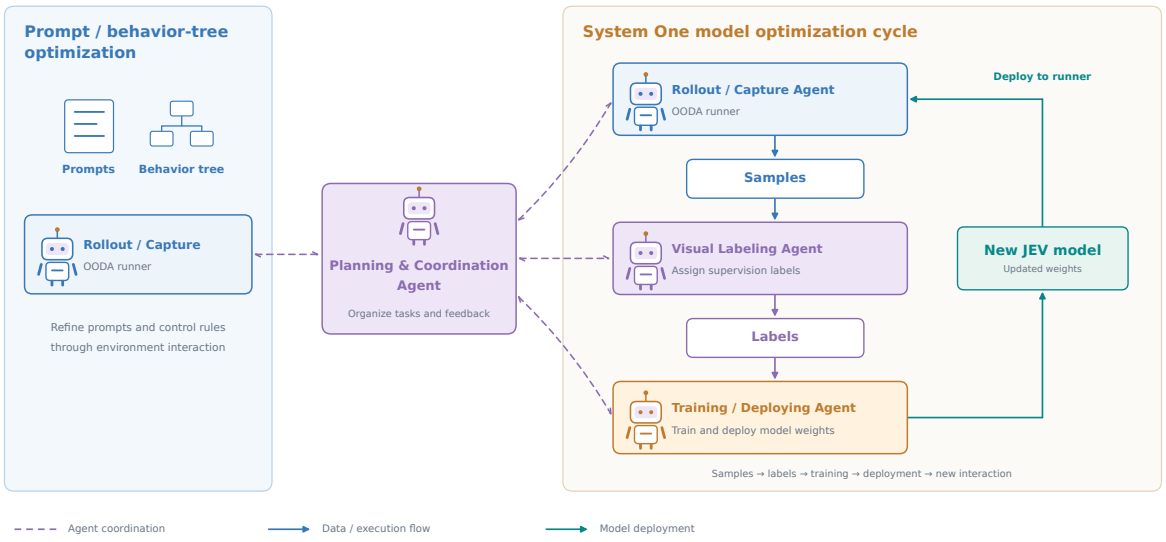


Figure 4: Agent roles in controller revision and model adaptation.

Figure 4 separates the controller-development path from the model-training path. Both paths share observations and deployment feedback through the coordinator.

2.2 Reasoning model and control interface

The reasoning model turns task knowledge into control rules and a supervision specification for recorded observations. Language-model-generated control programs have direct precedents in Code as Policies and Voyager [14, 15]. During online interaction, the controller obtains visual observations, calls JEV for structured judgments, applies decision rules, and submits actions

through the environment's control interface. Task knowledge is represented in both executable control rules and learned decision parameters.

From prior knowledge to task supervision

The framework can draw on web materials, informal human hints, reusable instructions or programs, and a small number of exploratory interactions with the live environment. Exploratory traces provide observations of environment states and input effects for subsequent analysis. A reasoning model can use these sources to define observable task features, candidate responses, and labels grounded in recorded observations. Control rules encode executable knowledge, while supervised adaptation consolidates selected judgments into JEV parameters. Supervision can therefore come from direct annotations or from a labelling protocol specified by the reasoning model, depending on the observable feedback. Reasoning-model supervision has precedents in distillation [7]; here the supervised targets are structured task judgments.

2.3 External muscle memory in JEV parameters

For an observation o_t and task question q, the JEV model produces a structured judgment z_t. The controller maps this judgment to an action a_t through

$$z_t = f_\theta(o_t, q), \qquad a_t = C_\phi(z_t). \tag{1}$$

JEV maps an observation and a task question to structured outputs in a single forward pass, allowing the controller to use its judgments without generating a reasoning trace at each step [3]. Here θ denotes the decision-model parameters and ϕ the controller rules and configuration. A pretrained visual backbone supplies representations, while a decision head defines the output interface. Task adaptation updates selected parameters, such as LoRA adapters [23], to consolidate supervised judgments into reusable responses. The output schema specifies the judgments required by the controller, rather than a fixed set of environment-specific actions.

2.4 Experience, supervision, and training

Training examples pair recorded observations with task-specific supervision. The reasoning model annotates selected observations or defines a labelling procedure that derives targets from recorded events and state changes. This converts task knowledge and interaction experience into training signals without requiring a reward function, expert operational demonstrations, or a parallel environment simulator. Labels derived from trajectories describe events under the recorded behaviour; they do not by themselves identify what would have happened under alternative actions.

The experience pool retains observations, actions, and available outcomes across interaction rounds. Sampling can emphasize errors, rare states, or task-relevant events, while validation supports checkpoint selection before redeployment. Supervised objectives train the model's structured outputs; class-weighted focal loss [24] is one option when label frequencies are imbalanced. The sampling distribution, training initialization, and validation partition are specified for each adaptation round. Repeated or adjacent observations must be distinguished from independent interaction experience when accounting for the data budget.

2.5 Recursive interaction and capability consolidation

An improvement round begins with experience generated by the deployed system and ends by returning an evaluated update to interaction. Writing r for the learning round, the proposed dependency is

$$\mathcal{T}_r = \text{Rollout}(E, f_{\theta_r}, C_{\phi_r}), \tag{2}$$
$$\mathcal{P}_{r+1} = \text{Aggregate}(\mathcal{P}_r, \mathcal{T}_r), \tag{3}$$
$$\mathcal{D}_{r+1} = \text{Supervise}_T(\mathcal{P}_{r+1}), \tag{4}$$
$$\theta_{r+1} = \text{Train}(\theta_{\text{init},r}, \mathcal{D}_{r+1}). \tag{5}$$

Here E is the environment, C_{ϕ_r} the controller at round r, T the reasoning model, $\mathcal{T}_r$ the interaction records, and $\mathcal{P}_r$ the experience pool. The initialization $\theta_{\text{init},r}$ is specified independently for each training run. Recursion is defined by the dependence of subsequent observations and supervision on the deployed system. After training and evaluation, the selected model re-enters OODA.

The accumulated experience pool supports data reuse across rounds, while evaluation determines whether an adapted model is suitable for deployment. Collecting labels on states visited by the current policy follows the motivation of dataset aggregation [22]; the supervision source and training objective here differ from expert-action imitation.

2.6 Model-only, controller-only, and joint improvement

Model parameters and controller rules define separate targets for improvement. Write the deployed system at round r as $S_r = (f_{\theta_r}, C_{\phi_r})$, where θ_r contains learned JEV parameters and ϕ_r denotes the controller's executable rules and configuration. The model-training equations above describe the model-only path; the other paths introduce controller revision guided by a reasoning model using the same recorded experience. This distinction separates parameter updates from code or feedback-based improvement, as illustrated by Voyager, Reflexion, and Self-Refine [15, 18, 25].

JEV-only improvement

Keep $\phi_{r+1} = \phi_r$, obtain supervision for recorded observations, and train θ_{r+1}. The objective is to improve task judgments while preserving the mapping from those judgments to actions.

OODA-only improvement

Keep $\theta_{r+1} = \theta_r$ and let the reasoning model revise C_{ϕ_r} from task knowledge and interaction traces. Updates can change decision rules, action timing, or setting-specific configuration without retraining JEV.

Joint improvement

Update both the JEV model and the OODA controller, and evaluate their combined behaviour after redeployment. If task questions or output schemas change, the controller and training labels must remain consistent with the updated interface. These update modes define separate interventions on the learned decision model and the controller.

Transfer across settings

A fine-tuned JEV model can be reused as a candidate decision module in another setting, with a different interface or task configuration. Holding the adapter fixed while changing the controller tests whether its judgments remain useful under the new configuration without additional JEV training. Keeping both fixed instead tests direct transfer of the complete system. If both are updated, the process constitutes joint adaptation rather than direct reuse. Transfer evaluation accordingly specifies the source and target configurations, parameter updates, controller changes, and additional interaction budget.

2.7 Online response and offline learning cost

Online response time includes observation, inference, decision, and action submission; offline learning incurs data collection, labelling, adaptation, and deployment costs. In the intended serial live-interaction regime, recorded experience is retained for reuse across training iterations. Prior knowledge informs the reasoning model and controller outside the time-critical execution path; the trained JEV supplies online judgments. Separating request latency from the complete observation-to-action interval is necessary when choosing the online response budget.

3 Experiments in Dynamic Visual Environments

Zenless Zone Zero and Black Myth: Wukong provide visual environments in which observations and action opportunities change during execution. Both implementations combine a 27B JEV model with an OODA controller. The ZZZ study examines model adaptation and subsequent controller revision across opponents. The Wukong study compares policies for sustained evasion and offensive action.

3.1 Visual judgments in Zenless Zone Zero

The ZZZ controller uses attack warnings, combat state, skill readiness, and character identity to select actions. LoRA [23] targets the attack-warning question while keeping the visual backbone frozen. The decision head and calibration remain unchanged. The training and validation sets contain 922 and 164 frames for `zzz-rsi-v1`, and 1,462 and 260 frames for `zzz-rsi-v2`. Validation negative log-likelihood (NLL) selects each checkpoint.

Figure 5 connects the deployed policy to the two adaptation rounds. Each round collects five episodes, labels attack warnings, and uses those examples to update JEV with LoRA. The behaviour tree converts the resulting judgments into skill use, movement, character switching, and attacks.

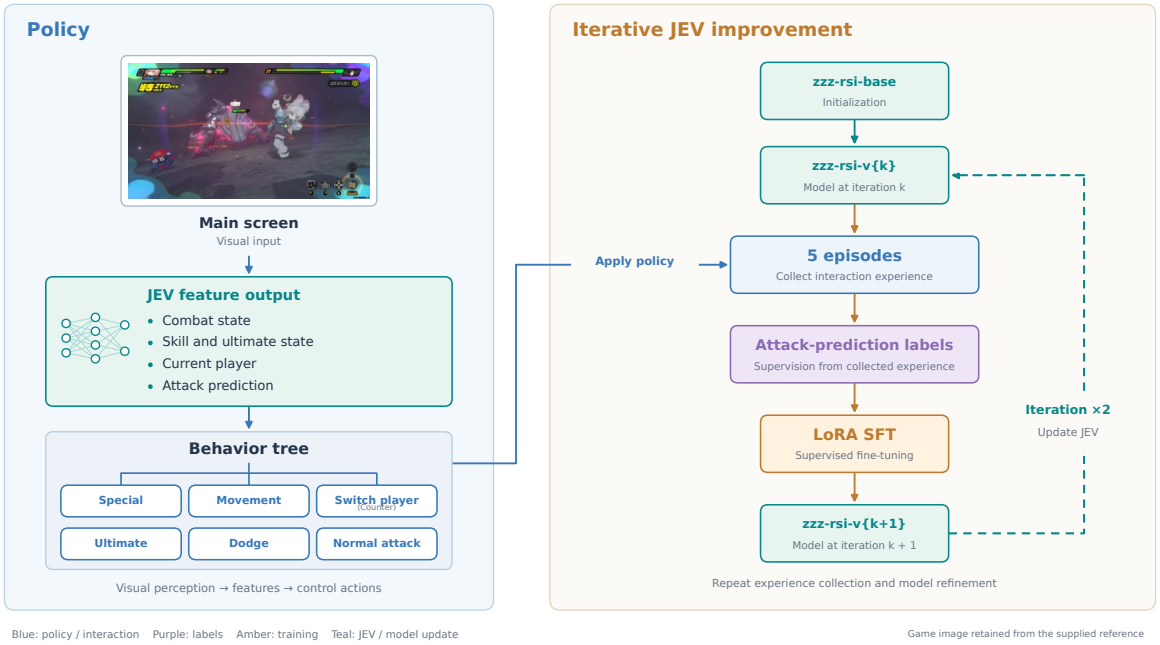


Figure 5: Policy execution and model adaptation in Zenless Zone Zero.

The adapted model returns to the same interaction process to collect the next round of experience. The validation comparison below measures changes in the attack-warning judgments that drive this policy.

Table 2 compares the models on a common set of 260 frames. The labels comprise 224 none, 11 yellow, and 25 red warnings. Balanced accuracy averages the three class recalls, giving each class equal weight. The validation reference differs from the online reference below; both are labelled `zzz-rsi-base` within their respective comparisons.

Table 2: ZZZ attack-warning validation.

Model	Top-1 (%)	Balanced accuracy (%)	NLL
Always none	86.15	33.33	N/A
`zzz-rsi-base`	63.85	44.52	0.6799
`zzz-rsi-v1`	79.23	74.87	0.4565
`zzz-rsi-v2`	81.15	77.98	0.4966

Top-1 agreement rises from 63.85% to 81.15%, while balanced accuracy rises from 44.52% to 77.98%. The constant-none predictor has higher overall accuracy but lower balanced accuracy than either adapted model. The distinction reflects improved recognition of the less frequent warning classes. Training and validation share source episodes, and the same validation set selects `zzz-rsi-v2`; these results do not estimate performance on independent episodes. Reusing a selection set for performance reporting can introduce selection bias [26].

3.2 Online control in Zenless Zone Zero

Each configuration runs five episodes with the same two-character team and controller under a 300 s limit. Table 3 reports completion and decision timing. Completion duration summarizes successful episodes, while t_{50} denotes the first time the opponent's imbalance gauge reaches 50%. The latency columns give the median and 95th percentile of logged decision-round durations.

Table 3: ZZZ online outcomes and decision latency.

Model	Wins	Median completion (s)	Median t_{50} (s)	p50/p95 (ms)
`zzz-rsi-base`	2/5	261.2	167.0	231.4/244.2
`zzz-rsi-v1`	5/5	236.2	26.0	229.1/249.4
`zzz-rsi-v2`	5/5	239.3	29.7	230.2/255.0

Both adapted configurations win all five episodes, compared with two wins for the reference. Their median decision latency remains approximately 230 ms. Figure 6 shows how opponent health changes during each episode. Crosses mark successful completion, and open circles mark unfinished episodes at the end of the record. The lower panel places the outcomes on the same time axis.

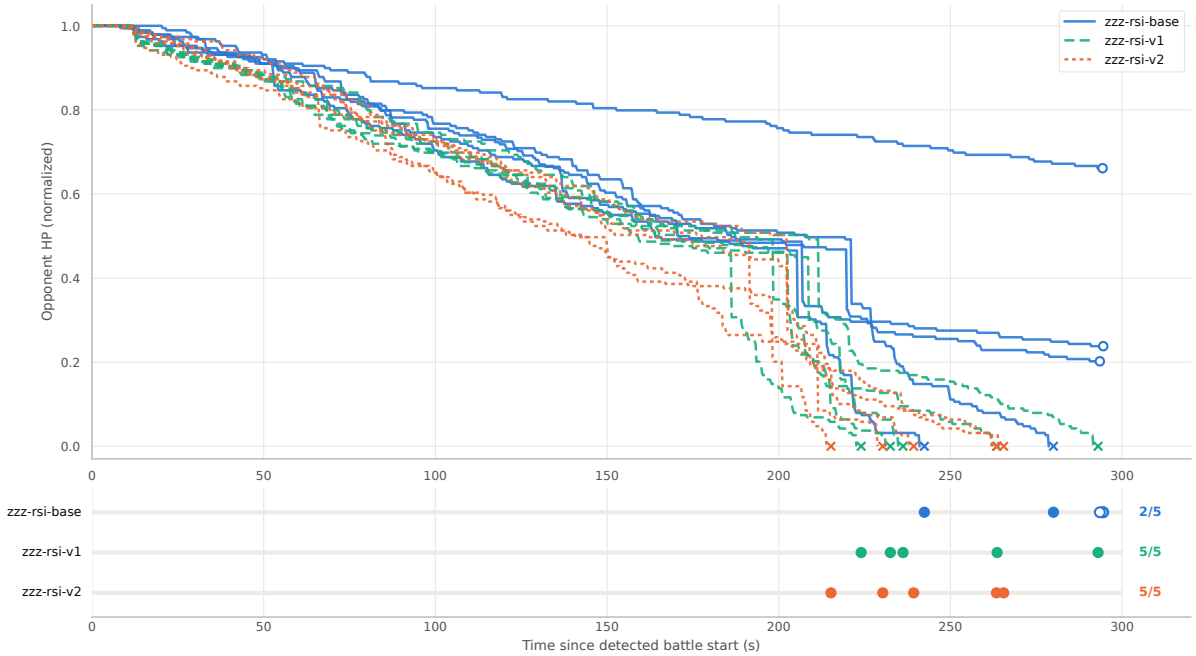


Figure 6: Opponent-health trajectories in fifteen ZZZ episodes.

The trajectories show differences in the pace of combat even when the final outcomes agree. With five unpaired episodes per configuration, they do not establish an ordering between `zzz-rsi-v1` and `zzz-rsi-v2`. The accompanying reference and adapted-model recordings show how visual judgments enter the action rules.

3.3 Controller revision across opponents

Controller revision changes the questions asked of JEV and the actions selected from its answers. The revised ZZZ configuration adjusts state descriptions and behaviour-tree rules, including action sequences and character switching after a parry. The strategies are fixed before evaluation on nine additional opponents under the P2 configuration. These evaluations comprise 39 attempts, of which 26 complete the task. The display names `zzz-rsi-v2` and `zzz-rsi-v3` identify the earlier and revised controller configurations in this comparison; the study does not isolate a fixed-weight controller intervention.

Figure 7 compares mean completion times on opponents with at least one completed episode. Table 4 gives the corresponding values. YeShiyuan is a separate comparison with five earlier and

three revised-controller episodes, outside the nine-opponent attempt count. All means condition on completion, so an unfinished attempt is not assigned an artificial completion time.

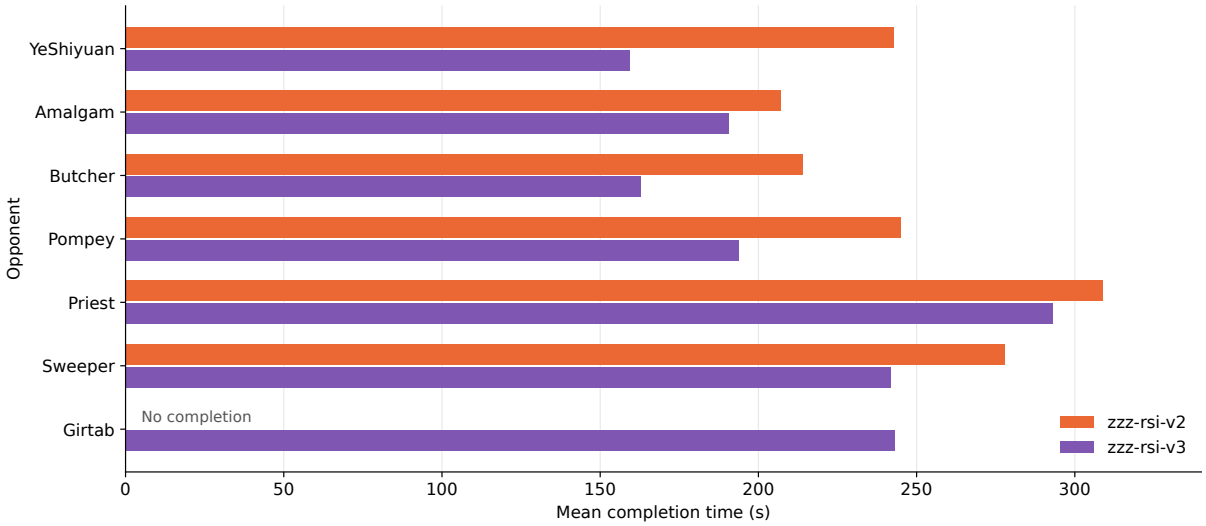


Figure 7: Mean completion time under two ZZZ controller configurations.

Table 4: Completion times by opponent.

Opponent	`zzz-rsi-v2` mean (s)	`zzz-rsi-v3` mean (s)
YeShiyuan	242.7	159.3
Amalgam	207.2	190.8
Butcher	214.0	163.0
Pompey	245.0	194.0
Priest	309.0	293.0
Sweeper	278.0	242.0
Girtab	N/A	243.0

For Amalgam, each configuration completes five episodes, with mean time decreasing by 16.4 s from 207.2 to 190.8 s. The revised controller completes three YeShiyuan episodes in 162, 165, and 151 s, averaging 159.3 s. For Priest, completed episodes take 309 and 293 s under the earlier and revised configurations, respectively; each also has one unfinished attempt. The earlier controller does not complete either Girtab attempt, whereas the revised controller completes two in 257 and 229 s. Hunter, Twins, and Bringer have no completed episodes in this evaluation. Sample counts and completion subsets differ between opponents, so these means describe the observed runs rather than a uniform improvement rate.

3.4 Evasive control in Black Myth: Wukong

The Wukong model predicts the remaining time before damage from the current observation and four historical frames. The controller uses that prediction to time a dodge.

Figure 8 shows how recent observations, predicted time to damage, and interaction data enter the adaptation process. Its action branches describe the broader policy family; the evasive evaluation below enables only the evasive actions.

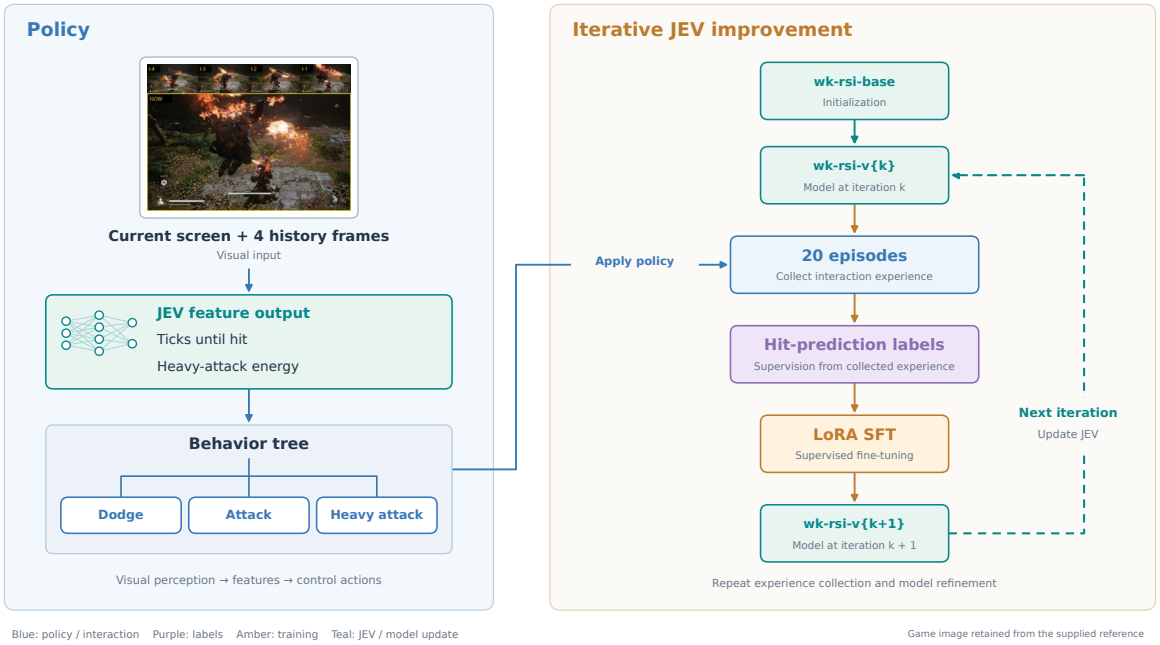


Figure 8: Policy execution and model adaptation in Black Myth: Wukong.

The illustrated training cycle collects interactions, constructs hit-prediction labels, and updates JEV before redeployment. The evasive policy issues no attacks, heavy attacks, or healing actions. Table 5 compares recorded episode durations with a stationary reference that takes no active actions.

Figure 9 plots the group means and medians, with episode counts alongside each row. Vertical marks denote medians; the stationary reference has no median reported in this comparison.

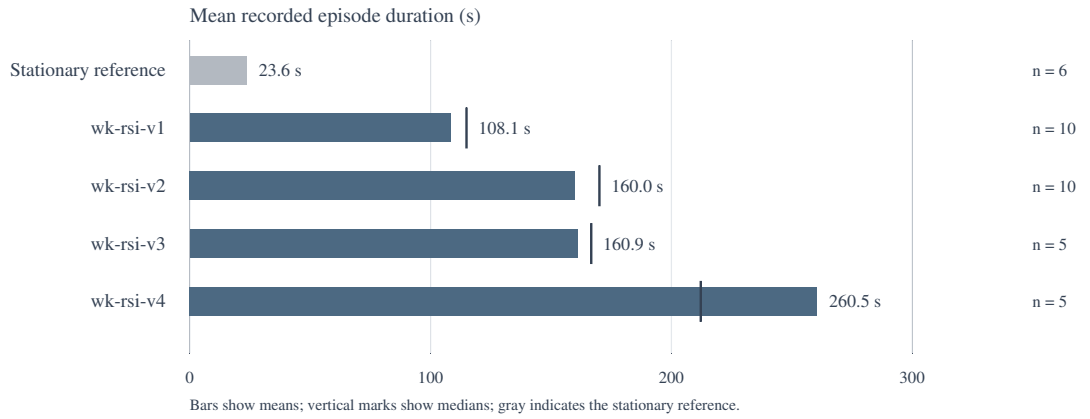


Figure 9: Recorded durations under the Wukong evasive policy.

The gap between the mean and median summarizes differences within each set of recorded episodes. Table 5 provides the numerical values and separates the experimental batches.

Under the later configuration, mean duration increases from 160.9 s for `wk-rsi-v3` to 260.5 s for `wk-rsi-v4`, a difference of approximately 62%. The earlier v1/v2 batches use a 350 ms action cooldown, while the later v3/v4 batches use 500 ms. Recorded durations include the time required to detect termination and do not identify an exact time of death. The batches therefore remain separate comparisons. Figure 10 shows the changing visual conditions in which these responses are selected.

Table 5: Wukong evasive-policy durations.

Model	Episodes	Mean (s)	Median (s)
Stationary reference	6	23.6	N/A
`wk-rsi-v1`	10	108.1	114.9
`wk-rsi-v2`	10	160.0	170.1
`wk-rsi-v3`	5	160.9	166.7
`wk-rsi-v4`	5	260.5	212.2



(a) (b)

Figure 10: Visual observations during evasive control in Black Myth: Wukong.

Both observations contain rapid motion and occlusion near the opponent. Recent frames provide information about how the action has developed, while the current frame constrains the timing of the response. Task completion also requires choosing when to attack, motivating a separate offensive policy.

3.5 Offensive control in Black Myth: Wukong

The offensive policy coordinates dodging, attacks, and healing. Win rate measures task completion, and mean winning-episode duration describes the time taken by successful attempts. Table 6 reports the five model configurations in two experimental batches. The first three configurations each run ten episodes; the remaining two each run fifteen.

Table 6: Wukong offensive-policy outcomes.

Model	Episodes	Win rate (%)	Mean duration (s)	Winning mean (s)
`wk-rsi-base`	10	0.0	21.9	N/A
`wk-rsi-v1`	10	20.0	74.1	139.7
`wk-rsi-v2`	10	30.0	89.5	122.4
`wk-rsi-v3`	15	60.0	139.7	154
`wk-rsi-v4`	15	66.7	125.3	131

The base model wins none of its ten episodes, while every adapted configuration has at least one win. The `wk-rsi-v4` configuration wins ten of fifteen episodes, with a mean winning-episode duration of 131 s. The two batches use different controller settings, and successful episodes form different subsets of the attempted runs. The values do not isolate the effect of model adaptation across all five configurations. The evasive-policy recording and offensive-policy recording illustrate the action timing behind these aggregate outcomes.

4 Design Properties and Limitations

4.1 Learning and reuse

The framework organizes capability formation around targeted practice guided by informal language. Prior knowledge identifies what to observe, and interaction supplies task-specific experience. Adaptation consolidates the resulting distinctions into a decision model.

Learning from limited interaction

Retaining observations across rounds allows the system to reuse costly experience instead of requiring a new environment instance for every training update. Supervision can focus on errors and informative states, making the approach suitable for settings in which replication, reset, and parallel rollout are unavailable. The interaction budget required to acquire a particular capability remains task dependent.

A compact online decision path

The online controller uses structured JEV judgments and executable rules, while extended reasoning and parameter adaptation take place outside the response-critical path. Learning can encode fine perceptual distinctions in parameters without repeatedly expanding their verbal specification at every step. Observation processing, serving, and action execution still determine the achievable response frequency.

Using prior knowledge to guide exploration

Web retrieval, informal human hints, and exploratory actions provide complementary sources of information about an unfamiliar environment. The reasoning model turns these sources into initial questions and control rules, then revises them using recorded feedback. This process directs practice toward relevant distinctions rather than relying exclusively on undirected trial and error. Using prior knowledge to construct executable skills has related precedents in Code as Policies and Voyager [14, 15].

Targeted model and controller updates

The separation between JEV and the controller makes the object of an update explicit. When the judgments are useful but their mapping to actions is inadequate, the controller can change while the model remains fixed. When perception is inadequate, adaptation can target JEV while preserving the control rules. Joint revision is available when the task requires a different judgment interface.

Reusing the OODA controller

Explicit control rules make dependencies between observations, decisions, and actions inspectable. Adapting an execution interface or revising a subset of rules can reuse the surrounding control structure in a new setting. The extent of that reuse depends on which task assumptions and action semantics remain valid.

Reusing JEV judgments

A structured decision interface allows an adapted model to be combined with revised questions and control rules. When its learned distinctions remain applicable, the same parameter set can support a different controller without additional model training. Such reuse does not establish that adaptation preserves every unrelated capability; new questions and changed observation distributions require validation. Policy-dependent state distributions are also central to the motivation for dataset aggregation [22].

An interpretable learning process

The external-muscle-memory analogy connects deliberate task understanding and targeted practice under informal verbal guidance. It describes how knowledge becomes available for action through learning, rather than treating fast inference alone as a sufficient account of skill acquisition. The explicit interaction and adaptation loops make the sources of subsequent updates identifiable.

No requirement for expert operational demonstrations

Training observations can be collected by the agent itself and labelled through reasoning or event-based protocols. The framework therefore does not require a human expert to perform the task to generate demonstrations. Public text and images remain useful sources of knowledge, distinct from a dataset of expert actions.

4.2 Scope and deployment constraints

Task-specific generalization

Learned judgments depend on the visual conditions and state distribution represented in the experience pool. A change in environment can require revised prompts, control rules, or additional interaction and adaptation. Reusing an interface or an adapter is a mechanism for transfer, rather than a guarantee of unchanged performance across settings.

Dependence on external priors

Dispensing with expert operational demonstrations does not eliminate dependence on human knowledge. Initial questions and control rules rely on descriptions of task structure, available actions, and relevant state changes. Missing or inaccurate priors can misdirect both exploration and supervision, making the initial policy and subsequent learning less effective.

Hardware and serving requirements

Low-latency execution depends on model size, observation processing, communication, and competition for compute resources. Separating environment rendering and inference across GPUs can reduce contention but increases deployment cost. Resource-limited deployments may require different model sizes, serving arrangements, or response budgets. The framework does not impose a universal GPU count or a fixed latency bound.

Supervision and update quality

A recursive learning loop does not ensure that every update improves control. Reasoning-model annotations can be incorrect, trajectory-derived labels depend on the actions that were taken, and adjacent observations provide correlated supervision. Model selection can itself overfit a finite validation set [26]. Future evaluations should separate checkpoint selection from testing on distinct interaction episodes and retain previous models for comparison.

5 Conclusion

System-One RSI connects task understanding to executable capability through targeted practice under informal verbal guidance. A reasoning model supplies task knowledge and control rules; JEV parameters retain learned perceptual judgments as external muscle memory for online OODA execution. The separation between these components supports targeted model updates, controller revision, and reuse when the relevant task assumptions remain valid. The framework addresses capability formation in dynamic environments where expert demonstrations and parallel simulation may be unavailable. The ZZZ and Wukong deployments illustrate the connection between learned judgments and online action, while small samples and different experimental settings limit comparative conclusions. Its effectiveness depends on the quality of prior knowledge and supervision, the coverage of collected experience, and the resources available for deployment.

References

[1] B. Y. Lin, et al. SwiftSage: A Generative Agent with Fast and Slow Thinking for Complex Interactive Tasks. arXiv:2305.17390, 2023. https://arxiv.org/abs/2305.17390

[2] G. L. Bailo. System Switch: When should a fast decision model stop and think? In *arXiv:2610.09683*, 2026. https://arxiv.org/abs/2610.09683

[3] TypeSafe AI. Introducing System One Models & Jev. TypeSafe AI Blog, September 2026. https://typesafe.ai/blog/introducing-system-one-models-and-jev

[4] Q. Li, X. Wang. Jeva: System One models from first principles. In *arXiv:2610.18773*, 2026. https://arxiv.org/abs/2610.18773

[5] G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. arXiv:1503.02531, 2015. https://arxiv.org/abs/1503.02531

[6] A. A. Rusu, et al. Policy Distillation. arXiv:1511.06295, 2015. https://arxiv.org/abs/1511.06295

[7] C.-Y. Hsieh, et al. Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes. *Findings of ACL*, pp. 8003–8017, 2023. https://aclanthology.org/2023.findings-acl.507/

[8] K. A. Ericsson, R. T. Krampe, C. Tesch-Römer. The Role of Deliberate Practice in the Acquisition of Expert Performance. *Psychological Review*, 100(3):363–406, 1993. https://doi.org/10.1037/0033-295X.100.3.363

[9] D. Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[10] M. Ahn, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv:2204.01691, 2022. https://arxiv.org/abs/2204.01691

[11] C. Berner, et al. Dota 2 with large scale deep reinforcement learning. In *arXiv:1912.06680*, 2019. https://arxiv.org/abs/1912.06680

[12] G. Dulac-Arnold, D. Mankowitz, T. Hester. Challenges of Real-World Reinforcement Learning. arXiv:1904.12901, 2019. https://arxiv.org/abs/1904.12901

[13] S. Levine, A. Kumar, G. Tucker, J. Fu. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. arXiv:2005.01643, 2020. https://arxiv.org/abs/2005.01643

[14] J. Liang, et al. Code as Policies: Language Model Programs for Embodied Control. arXiv:2209.07753, 2022. https://arxiv.org/abs/2209.07753

[15] G. Wang, Y. Xie, et al. Voyager: An open-ended embodied agent with large language models. In *arXiv:2305.16291*, 2023. https://arxiv.org/abs/2305.16291

[16] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, G. Neubig. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. arXiv:2107.13586, 2021. https://arxiv.org/abs/2107.13586

[17] T. Xie, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024. https://arxiv.org/abs/2404.07972

[18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023. https://arxiv.org/abs/2303.11366

[19] M. Robeyns, M. Szummer, L. Aitchison. A Self-Improving Coding Agent. arXiv:2504.15228, 2025. https://arxiv.org/abs/2504.15228

[20] J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. arXiv:2505.22954, 2025. https://arxiv.org/abs/2505.22954

[21] K. Irie, I. Schlag, R. Csordás, J. Schmidhuber. A Modern Self-Referential Weight Matrix That Learns to Modify Itself. In *ICML*, 2022. https://arxiv.org/abs/2202.05780

[22] S. Ross, G. J. Gordon, J. A. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *AISTATS*, PMLR 15:627–635, 2011. https://proceedings.mlr.press/v15/ross11a.html

[23] E. J. Hu, et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021. https://arxiv.org/abs/2106.09685

[24] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár. Focal loss for dense object detection. In *ICCV*, 2017. arXiv:1708.02002. https://arxiv.org/abs/1708.02002

[25] A. Madaan, et al. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651, 2023. https://arxiv.org/abs/2303.17651

[26] G. C. Cawley, N. L. C. Talbot. On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. https://www.jmlr.org/papers/v11/cawley10a.html